



TASKHUB

Scenario Applicativo

TaskHub è un'applicazione web per la gestione di attività personali e condivise.

Gli utenti possono:

- Registrarsi ed effettuare il login.
- Creare task giornaliere con titolo, priorità e commento.
- Modificare task **solo nel giorno in cui sono state create**.
- Creare gruppi di lavoro condivisi.
- Unirsi ad un gruppo tramite codice.
- Visualizzare ed eseguire task dei gruppi.
- Eliminare task solo se è il giorno corrente.
- Uscire dai gruppi; solo il creatore può eliminarli definitivamente.
- Sincronizzare tutte le operazioni in **tempo reale** tramite Socket.IO.

L'applicazione è ottimizzata per uso giornaliero con un'interfaccia chiara e semplice.

Architettura dell'Applicazione

L'app è strutturata con un **architettura 3-tier**:

Frontend (React)

- Single Page Application (SPA)
- Comunicazione con API REST via fetch
- Componenti modulari (TaskColumn, TaskItem, TaskForm, ecc.)
- Routing client-side (Home → Dashboard → InfoPersonali)

Backend (Node.js + Express)

- API REST con routing separato in moduli
- Middleware per autenticazione JWT
- Socket.IO integrato nel server Express per modifiche in tempo reale
-

Database (Mongoose+MongoDB)

- Storage NoSQL per utenti, gruppi e task
- Relazioni tramite ObjectId
- Indici su email utente e codice gruppo

Modello dei Dati

User

```
{
  "_id": "ObjectId",
  "email": "String",
  "password": "String (hashed)",
  "age": Number,
  "birthplace": "String"
}
```

Group

```
{
  "_id": "ObjectId",
  "name": "String",
  "code": "String (codice univoco)",
  "createdBy": "ObjectId(User)",
  "members": ["ObjectId(User)"]
}
```

Task

```
{
  "_id": "ObjectId",
  "title": "String",
  "priority": "low | medium | high",
  "comment": "String",
  "completed": Boolean,
  "date": "YYYY-MM-DD",
  "userId": "ObjectId(User)",
  "groupId": "ObjectId(Group) | null"
}
```

Documentazione API (Backend)

User

Metodo	Endpoint	Descrizione
POST	/user/register	Registra utente
POST	/user/login	Login utente
GET	/user/me	Dati utente loggato
DELETE	/user/me	Elimina utente

Tasks

Metodo	Endpoint	Descrizione
GET	/tasks	Lista task dell'utente + gruppi
POST	/tasks	Crea task
PUT	/tasks/:id	Modifica task (solo oggi)
DELETE	/tasks/:id	Elimina task (solo oggi)

Regola di sicurezza lato server:

- Una task può essere modificata/eliminata solo se task.date === oggi.

Groups

Metodo	Endpoint	Descrizione
GET	/groups	Lista gruppi a cui l'utente appartiene
POST	/groups/create	Crea un gruppo
POST	/groups/join	Unirsi tramite codice
POST	/groups/leave	Lascia un gruppo
DELETE	/groups/:id	Elimina DEFINITIVAMENTE (solo owner)

Socket.IO

- task:new
- task:update
- task:delete
- group:deleted
- joinRooms

Componenti React (Frontend)

HomePage

- Login e registrazione
- Presentazione dell'app
- Accesso alla dashboard

DashboardPage

- Mostra task e gruppi
- Suddivise in colonne dinamiche
- Gestisce Socket.IO
- Contiene logica per:
 - addTask
 - toggleTask
 - editTask
 - deleteTask
 - joinGroup
 - createGroup
 - leaveGroup
 - deleteGroup

TaskColumn

- Una colonna di task
- Contiene:
 - titolo
 - lista task
 - form aggiunta task

TaskItem

- Visualizzazione singola task
- Toggle per completamento
- Pulsanti modifica/elimina
- Indicatore di priorità

TaskForm

- Form selezione:
 - titolo
 - priorità
 - descrizione

InfoPersonali

- Mostra dati utente recuperati da /user/me con possibilità di modificare dati o eliminare account.

Diagrammi UML dei casi d'uso e sequenze

